

Introducción a sistemas operativos

Componentes de un sistema operativo

1. Usuarios

- Representan a los procesos o sesiones de usuario en ejecución.
- Corresponden a las instancias de programas que interactúan con el sistema.
- Varias instancias pueden funcionar simultáneamente y solicitar servicios al SO.

2. Herramientas, aplicaciones y programas

- Herramientas de desarrollo y utilidades: compilador, ensamblador, editor de texto, intérprete, depurador...
- Aplicaciones de usuario: navegadores, hojas de cálculo, sistemas de bases de datos, servidores web, gestores de impresión, demonios de sistema, etc.
- Todas dependen de las llamadas al sistema para acceder a recursos y están construidas sobre el kernel del SO.

3. Sistema Operativo

- Kernel: capa central que gestiona directamente la CPU, la memoria y los dispositivos de E/S.
- Programas del sistema: servicios en segundo plano y utilidades cercanas al kernel que gestionan tareas avanzadas.
- Proporciona interfaces estables que abstraen la complejidad de hardware y garantizan la seguridad y eficiencia en el acceso a los recursos.

4. Hardware

- Componentes físicos: CPU, RAM, disco/SSD, tarjetas de red, periféricos de E/S.
- Constituye la base sobre la que opera el kernel; todas las demás capas dependen de estos dispositivos.
- El kernel oculta los detalles eléctricos y de diseño de los dispositivos, ofreciendo una visión uniforme y simplificada al resto del sistema.

Módulos de un SO

Dentro del nivel de kernel, el sistema operativo se organiza en varios módulos o subsistemas, cada uno encargado de gestionar un aspecto fundamental de la máquina. Los más importantes son:

1. Sistema de ficheros

- **Función principal:** Proporcionar una estructura lógica para almacenar, organizar y recuperar datos en dispositivos de almacenamiento.
- **Responsabilidades:**
 - **Estructura de directorios:** crear, renombrar y eliminar ficheros y carpetas.
 - **Asignación de espacio:** determinar dónde y cómo se guardan los bloques de datos en el dispositivo físico.
 - **Control de acceso:** aplicar permisos y listas de control para proteger la integridad y confidencialidad de los datos.
 - **Caché de ficheros:** mantener en memoria bloques de disco recientemente accedidos para mejorar el rendimiento de lectura/escritura.

2. Gestión de memoria

- **Función principal:** Controlar el uso de la memoria principal (RAM) garantizando que cada proceso disponga del espacio que necesita de forma aislada y eficiente.
- **Responsabilidades:**
 - **Asignación de memoria:** repartir bloques de memoria a procesos al crearlos o ceder sus datos.
 - **Protección y aislamiento:** evitar que un proceso acceda a la memoria de otro, protegiendo la estabilidad y la seguridad del sistema.
 - **Memoria virtual:** mapear direcciones lógicas a direcciones físicas mediante paginación o segmentación, posibilitando el uso de más memoria que la físicamente disponible.
 - **Swapping o intercambio:** mover páginas de memoria entre RAM y disco cuando la memoria física escasea.

3. Gestión de procesos

- **Función principal:** Crear, programar y coordinar la ejecución de los procesos e hilos que conforman las aplicaciones y servicios del sistema.
- **Responsabilidades:**
 - **Creación y terminación:** invocar `fork()`, `exec()` o sus equivalentes para iniciar nuevos procesos, y liberar recursos al finalizar.
 - **Planificador (scheduler):** decidir qué proceso se ejecuta en cada instante, cuándo ceder la CPU y cómo repartir el tiempo entre procesos.
 - **Sincronización y comunicación:** proporcionar mecanismos para coordinar y compartir datos entre procesos.
 - **Estados de proceso:** gestionar las transiciones entre estados según eventos.

4. Gestión de entrada/salida

- **Función principal:** Facilitar el intercambio de datos entre el computador y sus periféricos.
- **Responsabilidades:**
 - **Drivers:** módulos específicos que conocen el protocolo de cada dispositivo físico y traducen solicitudes genéricas del SO a comandos de hardware.
 - **Colas de E/S y buffering:** almacenar temporalmente datos en búferes o colas para compensar diferencias de velocidad entre CPU/memoria y dispositivos.
 - **Modo bloqueante y no bloqueante:** ofrecer interfaces para que las aplicaciones puedan elegir entre esperar a que termine la operación o continuar con otras tareas.
 - **Notificaciones e interrupciones:** utilizar interrupciones hardware para informar al kernel de la finalización de una operación de E/S y despertar procesos bloqueados.

Jerarquía de memoria

En un sistema informático moderno existen diversos tipos de almacenamiento, organizados en una jerarquía según dos criterios fundamentales:

- **Capacidad de almacenamiento:** la cantidad de datos que puede contener cada tipo de memoria.
- **Tiempo de asociamiento:** la rapidez con la que se puede acceder a los datos almacenados.

1. Almacenamiento primario

Esta categoría incluye los tipos de memoria más rápidos que permiten acceder casi instantáneamente a los datos, pero cuya capacidad suele ser muy limitada y su contenido se pierde al apagar el equipo, es **volátil**

- **Registros**
 - Se encuentran dentro de la CPU y tienen acceso instantáneo y se usan para almacenar los datos e instrucciones que el procesador está utilizando en ese preciso momento.
 - Capacidad extremadamente reducida.
- **Memoria caché**
 - También está muy cerca del procesador, y puede estar dividida en tres niveles (L1, L2 y L3).
 - Almacena copias de datos frecuentemente usados para acelerar el acceso.
 - Es más grande que los registros, pero aún muy limitada.
 - Tiempo de acceso muy bajo, aunque mayor que el de los registros.
- **Memoria principal, RAM**
 - Es la memoria general de trabajo del sistema.
 - Se tarda más en acceder que la caché pero su capacidad es mayor.

- Es volátil.
- Usada para almacenar temporalmente los programas y datos en ejecución.

2. Almacenamiento secundario

Este nivel incluye medios que permiten almacenar datos de forma **persistente**, pero son más lentos que la memoria principal.

- **Memoria no volátil**
 - Incluye memorias como Flash, EEPROM o ROM.
 - Son rápidas comparadas con discos duros, pero más lentas que la RAM.
 - Se usan en BIOS, almacenamiento USB, SSD, etc.
- **Discos duros y unidades de estado sólido (HDD y SSD)**
 - Alta capacidad de almacenamiento
 - Persistentes

3. Almacenamiento terciario

Incluye tecnologías diseñadas para **almacenamiento masivo a largo plazo**, generalmente en tareas de archivos o respaldo.

- **Discos ópticos (CD, DVD, Blu-ray)**
 - Medios de almacenamiento portátiles y duraderos.
 - Acceso más lento que los discos duros.
 - Capacidad limitada comparada con los SSD o HDD.
- **Cintas magnéticas**
 - Usadas principalmente para copias de seguridad y almacenamiento a muy largo plazo.
 - Extremadamente lentas en acceso, pero con capacidades muy altas.
 - Bajo coste por unidad de almacenamiento.

Relación entre capacidad y velocidad

Existe una relación inversa típica:

- **A mayor velocidad, menor capacidad:** los registros y la caché son muy rápidos, pero su tamaño es mínimo.
- **A mayor capacidad, menor velocidad:** discos, cintas y almacenamiento terciario son muy grandes, pero acceden a datos mucho más lentamente.

Volatilidad frente a persistencia

- **Memorias volátiles:** pierden su contenido al apagar el sistema. Se utilizan para tareas temporales y rápidas.
- **Memorias no volátiles:** conservan los datos sin necesidad de energía. Se utilizan para almacenamiento permanente.

Sistema de ficheros

En los sistemas operativos, los **ficheros** son unidades de almacenamiento de información. Pueden contener datos de usuario, como textos o imágenes, o datos del sistema, como configuraciones o programas. Para poder gestionar estos ficheros de manera eficiente, el sistema operativo utiliza una estructura organizada en varios niveles: **directorios, bloques de datos y bloques de índices**.

1. Directorios

Un **directorio** es una tabla o estructura que asocia nombres de ficheros con su información de localización. Cuando se crea un archivo, el sistema guarda su nombre en el directorio junto con una referencia que indica dónde se encuentra la información del archivo en el almacenamiento. Esta referencia no apunta directamente a los datos, sino a un **bloque de índice**.

2. Bloques de Índice

Un **bloque de índice** es una estructura intermedia que almacena las direcciones de los **bloques de datos** reales dónde está guardado el contenido del archivo. Este bloque actúa como una lista de punteros. Es decir, en lugar de que el directorio conozca directamente los bloques que forman el archivo, delega esta tarea a un bloque de índice. Esto facilita el manejo de archivos fragmentados y mejora la organización del sistema de archivos.

3. Bloques de Datos

Los **bloques de datos** son unidades físicas de almacenamiento en el disco. Cada bloque puede almacenar una parte del contenido de un archivo grande, por ejemplo, se divide en varios bloques, que no necesariamente están ubicados de forma contigua. El bloque de índice mantiene la relación entre el archivo y los bloques que lo componen.

Gestión de memoria

Memoria virtual

La **memoria virtual** es una técnica que permite que los programas se ejecuten como si dispusieran de una gran cantidad de memoria, aunque la **memoria física** disponible sea limitada. Esta memoria virtual combina **RAM** con **espacio en disco** para ampliar artificialmente la cantidad de memoria utilizable por los procesos.

El sistema operativo se encarga de gestionar esta memoria, dividiendo tanto la memoria física

como la memoria virtual en unidades más pequeñas llamadas **páginas**. Gracias a esta organización, los programas pueden cargarse y ejecutarse aunque no estén completamente en memoria física al mismo tiempo.

Swapping

El **swapping** es un mecanismo dentro de la memoria virtual que permite mover procesos o partes de procesos entre la **RAM** y el **almacenamiento secundario** según las necesidades del sistema.

****Funcionamiento del Swapping:**

1. Carga inicial:

- Supongamos que hay varios programas en ejecución.
- Estos programas están parcialmente cargados en la memoria principal, ocupando distintos bloques de memoria.

2. Memoria llena:

- Cuando la RAM se llena y se necesita cargar otro proceso, el sistema operativo no puede cargarlo directamente si no hay espacio.

3. Swap Out:

- El sistema operativo identifica un proceso inactivo o menos prioritario y **lo mueve temporalmente al disco**, a un espacio llamado **área de swap**.
- Esto libera espacio en la RAM para otros programas más activos.

4. Swap In

- Si el **programa B** vuelve a ser necesario, **se trae de nuevo desde el disco a la RAM**
- No necesariamente se carga en los mismos bloques que ocupaba antes.

Ventajas de la Memoria Virtual y el Swapping

- **Multiprogramación:** Permite ejecutar varios programas al mismo tiempo, incluso si no caben todos completamente en la RAM.
- **Aislamiento:** Cada proceso cree que tiene su propia memoria privada, lo que mejora la seguridad y la estabilidad del sistema.
- **Gestión eficiente:** El sistema operativo puede decidir qué partes de los programas mantener en RAM y cuáles mover al disco, optimizando el rendimiento general.

Multiprogramación

La **multiprogramación** es una técnica fundamental por los sistemas operativos para mejorar la eficiencia del uso del procesador y los dispositivos de entrada/salida.

¿Por qué es necesaria?

Un sólo usuario o programa no puede mantener ocupado al CPU y a los dispositivos de E/S todo el tiempo. Por ejemplo, cuando un programa necesita esperar para leer datos del disco o del teclado, el procesador queda inactivo si no hay otro trabajo disponible.

¿Cómo funciona?

- El sistema operativo mantiene **varios programas** en memoria al mismo tiempo.
- **Sólo uno de ellos se ejecuta activamente en la CPU**, mientras los demás esperan su turno o están en espera por operaciones de E/S.
- Cuando el trabajo actual necesita esperar, el sistema operativo **cambia a otro trabajo disponible**.
- Este mecanismo es controlado por **algoritmos de planificación de trabajos**.

De esta forma, **la CPU siempre tiene algo que hacer**, aprovechando al máximo los recursos del sistema.

Time Sharing

La **multiprogramación** evolucionó hacia el concepto de **time sharing**, o multitarea interactiva, especialmente útil para entornos con **usuarios múltiples**.

Características principales:

- La CPU **cambia rápidamente de un proceso a otro**, dando la ilusión de que todos se ejecutan simultáneamente.
- Esto permite que **varios usuarios interactúen con sus programas en tiempo real**, mientras están ejecutándose.
- El **tiempo de respuesta** esperado debe ser inferior a un segundo para mantener la sensación de interactividad.

Detalles técnicos

- Cada usuario tiene al menos **un proceso en ejecución en memoria**.
- Si hay **varios procesos listos para ejecutarse**, el sistema utiliza **planificación de CPU** para decidir cuál ejecutar a continuación.
- Si los procesos no caben todos en la memoria principal, se utiliza **swapping** para moverlos entre la memoria y el disco.
- Además, la técnica de **memoria virtual** permite ejecutar procesos aunque no estén completamente cargados en RAM.

System Calls

Los sistemas operativos proporcionan una interfaz entre el hardware y los programas de usuario. Esta interfaz se materializa en forma de **System calls**, que permiten a los programas acceder a servicios esenciales como gestión de archivos, procesos, comunicación, entre otros.

Interrupciones y modos de operación

Las llamadas al sistema implican una transición del **modo usuario al modo kernel**. Esta transición puede dar dos tipos de interrupciones

Característica	Interrupción por Hardware	Interrupción por Software
Origen	Dispositivo externo	Instrucción del programa
Generación	Señal eléctrica del hardware	Instrucción específica
Sincronización	Asíncrona	Síncrona
Contador de Programa	No se incrementa	Se incrementa normalmente
Prioridad	Generalmente baja	Alta en relación al código normal
Ejemplo	Pulsación de una tecla	Llamada al sistema para leer un archivo

Modo dual de operación

Para proteger al sistema operativo y a los demás procesos:

- El **hardware** implementa un **bit de modo** que distingue si se está ejecutando en **modo usuario o kernel**.
- Algunas instrucciones son **privilegiadas** y sólo se pueden ejecutar en modo kernel.
- Una **llamada al sistema cambia temporalmente al modo kernel**.
- Al retornar al programa de usuario, el sistema vuelve al modo usuario.

Evitar abuso de CPU: uso de temporizador

Para evitar que un proceso consuma el CPU indefinidamente:

- Se configura un **temporizador** antes de ejecutar el proceso.
- Cuando el contador llega a cero, **se genera una interrupción**.
- El sistema operativo puede entonces **interrumpir el proceso**, reasignar la CPU o incluso terminar el proceso si ha superado su tiempo asignado.

1. [Introducción a sistemas operativos](#)
2. [Ficheros](#)
3. [Memoria](#)
4. [Procesos](#)
5. [Entrada Salida](#)